MOMO Quant

Software Requirements Specification

Version: 1.0 Draft
Product Type: AI-assisted quantitative crypto futures trading platform
Primary Developer: Solo developer
Initial Build Style: Modular monolith with future service-splitting support

---

1. Purpose

MOMO Quant is an AI-assisted trading platform for crypto futures markets. It will support historical backtesting, replay mode, paper trading, and later live trading.

The system will use deterministic trading strategies supported by AI-based market classification, confidence scoring, parameter optimization, trade explanation, and performance analysis.

AI will not directly place trades. The risk engine and execution engine will control final trade approval.

---

2. Product Vision

MOMO Quant should become a trading research and automation platform, not just a simple trading bot.

The platform must allow the developer to:

- Test strategies on historical data.
- Watch bot decisions through replay mode.
- Run paper trading safely.
- Monitor strategy performance.
- Understand why trades were taken or rejected.
- Improve strategies through data and reporting.
- Move to live trading only after validation.

---

3. Initial Trading Scope

The first version will focus on:

- Crypto futures/derivatives.
- Top 5 high-liquidity coins.
- 3-minute and 5-minute trading charts.

- Higher timeframe confirmation using 15-minute candles.
- Maker-style limit orders in paper/live execution.
- Three initial strategies:
- Liquidity Sweep Strategy
- VWAP Mean Reversion Strategy
- EMA Pullback Strategy

The system should not require all strategies to agree. Instead, it will use market regime detection and weighted confidence scoring.

---

4. Trading Modes

The platform must support three modes:

4.1 Backtesting Mode

Uses historical data to test strategies.

Required features:

- Select symbol.
- Select date range.
- Select timeframe.
- Select strategy or strategy group.
- Include fees.
- Include realistic spread assumptions.
- Include simulated maker fills.
- Include slippage model.
- Produce full performance report.

4.2 Replay Mode

Replay mode allows the developer to visually inspect historical market behavior and bot decisions.

Required features:

- Play/pause historical market replay.
- Speed controls: 1x, 2x, 10x, 50x, 100x.
- Show candles.
- Show indicators.
- Show strategy signals.
- Show AI confidence.
- Show risk engine decisions.
- Show trade entry/exit simulation.

- Show “no trade” reasons.

Replay mode is critical because the developer must understand the bot’s behavior before trusting it.

4.3 Paper Trading Mode

Paper trading must use the same strategy, AI, risk, and execution logic as live trading.

Only the exchange connector should change.

Required features:

- Simulated balance.
- Simulated positions.
- Simulated limit orders.
- Partial fill simulation.
- Fees.
- Funding fee estimates.
- Trade journal.
- Daily PnL.
- Strategy performance tracking.

4.4 Live Trading Mode

Live trading will be added only after backtesting and paper trading are stable.

Required features:

- Exchange API key management.
- Maker/post-only limit orders.
- Reduce-only orders.
- Position monitoring.
- Emergency stop.
- Daily loss limit.
- Audit logs.
- Real-time alerts.

Live trading must be disabled by default.

---

5. Mode Switch Requirement

The platform must include a clear trading mode switch:

- Backtest
- Replay
- Paper
- Live

The current mode must be visible across the dashboard.

Live mode should require extra confirmation before activation.

The system must prevent accidental live trading.

---

6. Core Modules

MOMO Quant will use a modular monolith architecture for the first version.

Main modules:

- Authentication Module
- User Management Module
- Exchange Module
- Market Data Module
- Candle Storage Module
- Indicator Module
- Strategy Module
- AI Decision Module
- Risk Management Module
- Execution Module
- Backtesting Module
- Replay Module
- Paper Trading Module
- Live Trading Module
- Reporting Module
- Monitoring Module
- Notification Module
- Audit Logging Module
- Settings Module

Each module should be separated clearly in code so it can later become a separate service if needed.

---

7. Strategy Requirements

The platform must support strategy plugins.

Every strategy must expose:

- Name
- Description
- Supported timeframes
- Supported market regimes
- Required indicators
- Entry rules
- Exit rules
- Stop loss rules
- Take profit rules
- Confidence contribution
- Human-readable explanation
- Backtest compatibility

Initial strategies:

7.1 Liquidity Sweep Strategy

Detects stop-hunt style movement around swing highs and swing lows.

Useful for:

- Reversal setups.
- False breakout setups.
- Liquidity grab detection.

7.2 VWAP Mean Reversion Strategy

Detects price deviation from VWAP and possible reversion back toward fair value.

Useful for:

- Ranging markets.
- Overextended markets.
- Exhaustion moves.

7.3 EMA Pullback Strategy

Detects trend continuation entries after pullbacks into moving averages.

Useful for:

- Trending markets.
- Clean directional movement.
- Momentum continuation.

---

8. AI Requirements

The AI component must support the trading system but must not blindly control execution.

AI responsibilities:

- Market regime classification.
- Confidence scoring.
- Strategy weighting.
- Parameter optimization.
- Trade explanation.
- Anomaly detection.
- Performance pattern discovery.

AI must output structured decisions, for example:

- Market regime: Trending
- Confidence: 86%
- Preferred strategy: EMA Pullback
- Risk adjustment: Normal
- Trade allowed: Yes/No
- Explanation: Human-readable reason

AI must not bypass risk rules.

---

9. Market Regime Detection

The system must classify the current market condition.

Supported regimes:

- Trending
- Ranging
- Breakout
- Reversal
- High volatility
- Low volatility
- Choppy

- News spike / abnormal volatility

Strategies should only run when the current regime matches their allowed conditions.

---

10. Risk Management Requirements

The risk engine has final authority.

It can reject any trade even if all strategies and AI approve it.

Required rules:

- Maximum risk per trade.
- Maximum daily loss.
- Maximum weekly loss.
- Maximum open positions.
- Maximum exposure per symbol.
- Maximum correlated exposure.
- Maximum consecutive losses.
- Minimum confidence required.
- Maximum spread allowed.
- Maximum volatility allowed.
- Trading session filters.
- Emergency stop.

Stop losses should be ATR-based or structure-based, not fixed random percentages.

---

11. Execution Requirements

Execution must support maker-style limit orders.

Required execution features:

- Limit order creation.
- Post-only option.
- Reduce-only option.
- Order timeout.
- Cancel and replace.
- Partial fill handling.
- Fill tracking.
- Order status reconciliation.
- Simulated execution for paper/backtest modes.
- Real exchange execution for live mode later.

Maker-only execution can miss trades. The system must log missed trades separately so performance can be analyzed honestly.

---

12. Reporting Requirements

The platform must include complete reporting.

Reports must include:

- Total PnL.
- Daily PnL.
- Weekly PnL.
- Monthly PnL.
- Win rate.
- Profit factor.
- Expectancy.
- Average win.
- Average loss.
- Max drawdown.
- Fees paid.
- Funding cost.
- Best symbol.
- Worst symbol.
- Best strategy.
- Worst strategy.
- Performance by market regime.
- Performance by timeframe.
- Performance by trading session.
- Rejected trade reasons.
- Missed trade reasons.

Reports must be available for backtesting, paper trading, and live trading.

---

13. Monitoring Requirements

The system must monitor:

- Bot status.
- Current mode.
- Active symbols.
- Active strategies.
- Open positions.
- Open orders.
- WebSocket connection status.

- Exchange API status.
- Database health.
- Redis health.
- AI service health.
- Error count.
- Latency.
- Last market update time.
- Last successful execution time.

The dashboard must clearly show whether the bot is safe, degraded, or stopped.

---

14. Audit Logging Requirements

The platform must log every meaningful decision.

Log examples:

- Strategy produced signal.
- Strategy rejected setup.
- AI approved signal.
- AI rejected signal.
- Risk engine approved trade.
- Risk engine rejected trade.
- Order submitted.
- Order filled.
- Order partially filled.
- Order canceled.
- Order expired.
- Position opened.
- Position closed.
- Emergency stop triggered.
- Mode changed.
- Settings changed.

Every trade must be explainable after the fact.

---

15. Dashboard Requirements

Initial dashboard pages:

- Login
- Main Dashboard
- Bot Control
- Market Watch
- Strategy Signals

- Backtesting
- Replay Mode
- Paper Trading
- Trades
- Orders
- Positions
- Reports
- AI Analytics
- Risk Settings
- Exchange Settings
- Monitoring
- Logs
- User Management

The dashboard should prioritize clarity over visual complexity.

---

16. Technology Stack

Initial stack:

- Backend: .NET 8
- Frontend: React + Vite
- Styling: Tailwind CSS
- Database: MySQL
- Cache/queue: Redis
- AI Service: Python
- Charting: lightweight charting library
- Deployment: Docker Compose
- Authentication: JWT
- Realtime updates: SignalR or WebSocket

The first version should avoid unnecessary Kubernetes, Kafka, and microservice complexity.

---

17. Non-Functional Requirements

The system must be:

- Modular
- Testable
- Observable
- Configurable
- Secure
- Recoverable
- Maintainable by one developer
- Designed for future expansion

- Safe by default

Performance goals:

- Process market updates near real time.
- Support at least top 5 crypto symbols initially.
- Store historical candle data efficiently.
- Keep dashboard responsive.
- Recover from WebSocket disconnections.
- Prevent accidental live trading.

---

18. Security Requirements

Security requirements:

- JWT authentication.
- Role-based authorization.
- Encrypted exchange API keys.
- Audit logs for sensitive actions.
- Environment-based secrets.
- HTTPS in production.
- Live trading disabled by default.
- Confirmation required before live mode.
- Emergency stop accessible from dashboard.

---

19. Assumptions

The project assumes:

- The first developer is solo.
- The system should start simple but scalable.
- Crypto futures are the first supported market.
- Top coins are preferred due to liquidity.
- Maker orders are preferred to reduce trading costs.
- AI is advisory and analytical, not fully autonomous.
- Backtesting and paper trading must come before live trading.

---

20. Constraints

Constraints:

- Solo developer capacity.
- Limited initial trading research dataset.

- Exchange API rate limits.
- Maker orders may not fill.
- Backtest results may differ from live results.
- Crypto markets are volatile.
- AI optimization can overfit if not controlled.
- Real trading should not begin until paper trading proves stable.

---

21. Success Criteria

MOMO Quant v1 is successful if it can:

- Import historical candle data.
- Run backtests on selected symbols.
- Replay historical sessions with visible bot decisions.
- Run paper trading with realistic execution simulation.
- Log every trade and rejected signal.
- Generate detailed reports.
- Show live monitoring dashboard.
- Support the three initial strategies.
- Use AI for confidence scoring and market regime detection.
- Prevent unsafe or accidental live trading.

---

22. MVP Boundary

The MVP should not attempt to do everything.

MVP includes:

- One exchange integration.
- Top 5 symbols.
- 3m and 5m candles.
- 15m higher timeframe filter.
- Three strategies.
- Backtesting.
- Replay mode.
- Paper trading.
- Reporting.
- Monitoring.
- AI confidence scoring.
- Risk engine.
- Dashboard.

MVP excludes:

- Real money trading.
- Multi-exchange routing.
- Mobile app.
- Advanced portfolio optimization.
- Reinforcement learning.
- Strategy marketplace.
- Public subscription system.

---

23. Development Principle

The system should be built in this order:

1. Documentation
2. Architecture
3. Database design
4. API contracts
5. Core infrastructure
6. Historical data
7. Indicator engine
8. Strategy engine
9. Backtesting
10. Replay mode
11. Paper trading
12. Reporting
13. Monitoring
14. AI optimization
15. Live trading

No live trading should be implemented before backtesting and paper trading are reliable.